1. Consider the following piece of JavaScript:

1 point

```
function recLinSearch(array, n, x) {
    if (n<0) {
        return false;
    } else if (array[n] == x) {
        return true;
    } else {
        return MISSING;
    }
}

function LinSearch(array, x) {
    var n = array.length;
    return recLinSearch(array, n-1, x);
}
```

When completed, the JavaScript function LinSearch will recursively implement (using recLinSearch) the linear search algorithm on an array, looking for the element x. What should go in the place of MISSING to complete this function?

- ○
```
recLinSearch(array, n-1, x)
```

- ○
```
recLinSearch(array, n, x)
```

- ○
```
recLinSearch(array, x)
```

- ○
```
recLinSearch(array, n+1, x)
```

2. Consider the following piece of JavaScript:

1 point

```
function swap(array, i, j) {
    var x = array[j];
    array[j] = array[i];
    array[i] = x;
    return array;
}

function sort(array, r) {
    if (r < 1) {
        return array;
    }
    for (var j = 0; j < r; j++) {
        if (array[j+1] < array[j]){
            swap(array, j, j+1);
        }
    }
    return MISSING;
}

function bubbleSort(array) {
    var n = array.length;
    return sort(array, n-1);
}
```

When completed, the JavaScript function bubbleSort will recursively implement (using Sort) the bubble sort algorithm on an array. What should go in the place of MISSING to complete this function?

- ○
```
sort(array, r-1)
```

- ○
```
sort(array, r+1)
```

```
bubbleSort(array)
```

```
sort(array, r)
```

3. Consider the following piece of JavaScript:

1 point

```
function swap(array, i, j) {
    var x = array[j];
    array[j] = array[i];
    array[i] = x;
    return array;
}

function sort(array, r) {
    if (r < 1) {
        return array;
    }
    MISSING
    var j = r;
    while ((array[r] < array[j-1]) && (j > 1)) {
        swap(array, j, j-1)
        j--;
    }
    return array;
}

function insertionSort(array) {
    var n = array.length;
    return sort(array, n-1);
}
```

When completed the JavaScript function insertionSort will recursively implement (using Sort) the insertion sort algorithm on an array. What should go in the place of MISSING to complete this function?

```
sort(array, r+1);
```

```
sort(array, r);
```

```
sort(array, r-1);
```

```
insertionSort(array, r-1);
```

Coursera Honor Code Learn more

I, **TODOR SIMEONOV**, understand that submitting work that isn't my own may result in permanent failure of this course or deactivation of my Coursera account.*

You must select the checkbox in order to submit the assignment

Submit Save draft

Last saved on Mar 1, 4:21 PM CET

Like Dislike Report an issue